TRABAJO DE ARQUITECTURA DE SOFTWARE

ENTREGA FINAL

PRESENTADO POR
DORALIS MARTINEZ PASTRANA
JULIAN MORALES DE LA OSSA

PRESENTADO A
SONIA HUERFANO

CORPORACION UNIVERSITARIA REMINGTON
FACULTAD DE INGENIERIA

Contenido

Introducción 4
Drivers Arquitectónicos 6
Restricciones del Sistema 6
Requerimientos Funcionales Esenciales 7
Atributos de Calidad 8
Tabla Consolidada de Drivers 8
Escenarios de Calidad 11
Escenario 1 — Respuesta Reactiva ante Deslumbramiento (Rendimiento) 11
Escenario 2 — Comunicación Cooperativa V2V (Rendimiento / Interoperabilidad) 12
Escenario 3 — Disponibilidad ante Falla de Conectividad (Disponibilidad) 13
Diagramas C4: Contexto y Contenedores 14
Nivel 1 — Diagrama de Contexto del Sistema 14
Nivel 2 — Diagrama de Contenedores del Sistema 16
Justificación de Estilos Arquitectónicos 19
Estilo 1 — Publicador / Suscriptor (Pub/Sub) 19
Estilo 2 — Arquitectura en Capas (Layered Architecture) 20
Cuadro Comparativo de los Dos Estilos Arquitectónicos 22
Segunda entrega 24
Patrones Arquitectónicos Aplicados 24
Patrón Pub-Sub (Publicador / Suscriptor) 24
Patrón Circuit Breaker 24
Patrón Gateway (Puerta de Enlace de Configuración) 25
Diagrama C4 Nivel 3: Componentes 27
Descripción de Componentes por Contenedor 27
Diagrama C4 Nivel 3 — Representación Textual Estructurada 29
Contenedor: Nodo IoT — Vehículo A (ESP32) 30
Contenedor: Backend API (Node.js) 30
Contenedor: Dashboard (React.js) 30
Nota sobre el Diagrama draw.io 30
Implementación Funcional 31
Flujo de Datos de Punta a Punta 31
Estructura del Payload MQTT 32
Estructura de Tópicos MQTT 32
Simulación del Prototipo (Hardware o Simulado) 33
Verificación de Latencia (Escenario de Calidad AQ-01) 33
Tabla de Trade-offs Arquitectónicos 33
Resumen del Impacto de Trade-offs en Atributos de Calidad 35
Tercera parte 35
Documentación Completa — Views & Beyond 35
Vista de Módulos 35

Tabla de Elementos — Vista de Módulos 36
Decisión arquitectónica asociada a la Vista de Módulos 37
Vista de Componentes y Conectores (C&C) 37
Tabla de Elementos — Vista C&C 37
Decisión arquitectónica asociada a la Vista C&C 38
Vista de Despliegue 38
Tabla de Elementos — Vista de Despliegue 39
Decisión arquitectónica asociada a la Vista de Despliegue 40
Decisiones Arquitectónicas — Registro ADR Completo 40
Evaluación de Arquitectura — Checklist ATAM Simplificado 42
Checklist ATAM — Estado de Cumplimiento 42
Riesgos Arquitectónicos Identificados 43
Puntos Sensibles de la Arquitectura 44
Trade-offs Documentados 44
Despliegue con Docker Compose + Monitoreo 46
Servicios Contenerizados 46
Archivo docker-compose.yml 47
Métricas de Disponibilidad y Monitoreo 49
Comandos de Operación 50
CONCLUSIÒN 51
EXPERIENCIA PERSONAL. 52
REFERENCIAS 53

Introducción

El deslumbramiento vehicular en vías de doble sentido con iluminación deficiente representa uno de los factores de riesgo más frecuentes en accidentes de tránsito nocturno. Cuando un conductor enfrenta las luces altas de un vehículo en sentido contrario, su retina experimenta una saturación temporal que puede durar entre 2 y 8 segundos, lapso durante el cual la visibilidad se reduce drásticamente. A una velocidad de 80 km/h, un conductor puede recorrer entre 44 y 178 metros prácticamente a ciegas.

Frente a este problema, el presente trabajo propone el diseño arquitectónico de un Sistema IoT de Iluminación Inteligente para Seguridad Vial, denominado SafeLight IoT. El Sistema implementa dos capas de protección complementarias:

Capa	Nombre	Descripción	Dependencia
Capa 1	Protección reactiva autónoma	Sensor LDR/BH1750 mide la intensidad lumínica frontal y activa automáticamente un actuador local (espejo electrocrómático o filtro polarizado) para proteger al conductor.	Ninguna — funciona solo
Capa 2	Comunicación cooperativa V2V	Dos nodos ESP32 se comunican vía WiFi o BLE para detectarse mutuamente cuando se aproximan en sentido contrario y negociar la reducción coordinada de luces altas.	Requiere ambos vehículos equipados

Desde el punto de vista de la Arquitectura de Software, SafeLight IoT constituye un sistema embebido distribuido con requerimientos estrictos de tiempo real, alta disponibilidad y comunicación entre dispositivos. Su diseño arquitectónico debe balancear múltiples atributos de calidad: la latencia de respuesta debe ser inferior a 500 ms para la Capa 1, y la comunicación entre nodos debe establecerse en menos de 1 segundo para la Capa 2.

El presente documento estructura el análisis arquitectónico del sistema en cuatro componentes principales siguiendo la rúbrica del Reto 1: los drivers arquitectónicos que determinan las decisiones de diseño, los escenarios de calidad que formalizan los requerimientos de atributos, los diagramas C4 que describen la estructura del sistema, y la justificación de los estilos arquitectónicos seleccionados.

Drivers Arquitectónicos

Los drivers arquitectónicos son los factores que tienen el mayor impacto en la estructura y el comportamiento del sistema. Determinan qué decisiones de diseño deben tomarse y cuáles alternativas son viables. Para SafeLight IoT, los drivers se clasifican en tres categorías: restricciones del sistema, requerimientos funcionales esenciales y atributos de calidad críticos.

Restricciones del Sistema

Las restricciones son condiciones inamovibles que el sistema debe respetar sin negociación. Provienen del entorno de operación, las tecnologías disponibles, el presupuesto y las normativas aplicables. Para un sistema IoT vehicular, estas restricciones son particularmente exigentes debido al contexto de operación en tiempo real y en entornos físicos adversos.

ID	Tipo	Nombre	Justificación	Prioridad
DR-01	Restricción Hardware	Microcontrolador de bajo costo y bajo consumo	El prototipo debe construirse con componentes accesibles (Arduino UNO / ESP32). El ESP32 ofrece WiFi y BLE integrados, procesador dual-core 240MHz y bajo consumo energético.	Alta
DR-02	Restricción Conectividad	Comunicación inalámbrica de corto alcance	Los módulos de comunicación entre vehículos deben operar sin infraestructura de red fija.	Alta
DR-03	Restricción Energía	Alimentación desde batería vehicular o banco de energía	El sistema debe operar con 5V DC (USB) o 12V (batería vehicular). El consumo total no puede superar 500mA en modo activo.	Alta
DR-04	Restricción Ambiental	Operación en condiciones nocturnas y de vibración	Los sensores y actuadores deben tolerar temperaturas de -10°C a 60°C, vibraciones mecánicas y humedad variable.	Media
DR-05	Restricción Normativa	Compatibilidad con normas de señalización vial	La atenuación de luces vehiculares debe seguir parámetros establecidos por normas de tránsito aplicables.	Alta

Nota: Las restricciones de prioridad Alta son no negociables y deben satisfacerse completamente. Las de prioridad Media pueden mitigarse con consideraciones de diseño apropiadas.

Requerimientos Funcionales Esenciales

Los requerimientos funcionales esenciales son las capacidades que el sistema debe tener para cumplir su propósito fundamental. Se consideran 'esenciales' porque su ausencia haría que el sistema no cumpliera con su objetivo de seguridad vial.

ID	Tipo	Nombre	Descripción y Criterio de Aceptación	Prioridad
RF-01	Req. Funcional	Detección de deslumbramiento	El sistema debe medir continuamente la intensidad lumínica frontal mediante un sensor LDR o BH1750. Umbral de activación: > 1000 lux en modo nocturno.	**Crítica**
RF-02	Req. Funcional	Activación de actuador local (Capa 1)	Ante un evento de deslumbramiento detectado, el sistema debe activar el espejo electrocrómático o filtro polarizado en menos de 500ms.	**Crítica**
RF-03	Req. Funcional	Comunicación V2V (Capa 2)	Cuando dos nodos ESP32 se detecten mutuamente a una distancia inferior a 80 metros, deben negociar la reducción coordinada de luces altas.	Alta
RF-04	Req. Funcional	Restauración automática de luces	Una vez que los vehículos hayan completado el cruce (distancia superior a 80 metros), el sistema debe restaurar automáticamente las luces altas.	Alta
RF-05	Req. Funcional	Registro y publicación de eventos en broker MQTT	Cada evento de deslumbramiento detectado, activación de actuador, y comunicación V2V debe publicarse en el broker MQTT con timestamp y payload JSON.	Media
RF-06	Req. Funcional	Configuración de umbrales por software	Los umbrales de activación (lux para deslumbramiento, distancia para comunicación V2V) deben configurarse remotamente vía MQTT sin recompilar firmware.	Media

Atributos de Calidad

Los atributos de calidad (también llamados requerimientos no funcionales o preocupaciones de calidad) definen cómo debe comportarse el sistema más allá de sus funciones básicas. Para SafeLight IoT, la selección y priorización de atributos de calidad es determinante dado que opera en un contexto de seguridad activa donde los fallos pueden tener consecuencias físicas graves.

ID	Atributo	Medida / Escenario de Referencia	Justificación Arquitectónica
AQ-01	Rendimiento (Performance)	Tiempo de respuesta < 500ms en Capa 1	Desde la detección del umbral de deslumbramiento hasta la activación del actuador, el tiempo total debe ser inferior a 500ms para garantizar protección efectiva.
AQ-02	Disponibilidad (Availability)	Disponibilidad del sistema > 99% en operación nocturna	El sistema debe operar de forma ininterrumpida durante toda la jornada nocturna de uso vehicular.
AQ-03	Confiabilidad (Reliability)	Tasa de falsos positivos < 1% en detección	El sistema no debe activar el actuador de protección cuando no haya deslumbramiento real, para evitar interferencia con la conducción normal.
AQ-04	Seguridad (Safety)	No interferencia con la visibilidad mínima del conductor	El actuador nunca debe oscurecer completamente el campo visual del conductor ni activarse en condiciones de baja luminosidad natural.
AQ-05	Mantenibilidad (Maintainability)	Firmware actualizable remotamente (OTA)	Los parámetros del sistema deben poder actualizarse remotamente sin intervención física sobre el dispositivo.
AQ-06	Interoperabilidad (Interoperability)	Compatibilidad con protocolo MQTT estándar	El sistema debe publicar y suscribirse a tópicos MQTT con formato JSON estándar, compatible con cualquier broker MQTT 3.1.1 o 5.0.

Tabla Consolidada de Drivers

La siguiente tabla consolida todos los drivers identificados, mostrando su clasificación, impacto en el sistema y las decisiones arquitectónicas que condicionan. Esta visión de

conjunto permite al equipo de arquitectura priorizar decisiones de diseño según el impacto total de cada driver.

ID	Nombre	Categoría	Impacto Arq.	Decisión Arquitectónica Asociada
DR-01	Hardware de bajo costo	Restricción	**Muy Alto**	Usar ESP32/Arduino; evitar SBCs costosas (Raspberry Pi) para la lógica de sensado.
DR-02	Comunicación inalámbrica corto alcance	Restricción	**Muy Alto**	Protocolo WiFi Direct o BLE entre nodos; no depender de infraestructura externa.
DR-03	Alimentación por batería	Restricción	Alto	Diseño de bajo consumo; modo sleep entre lecturas; comunicación eficiente.
DR-04	Condiciones ambientales adversas	Restricción	Medio	Encapsulamiento de componentes; filtrado digital de señal del sensor LDR.
DR-05	Normativa vial	Restricción	Alto	Lógica de actuador con límites de atenuación; validación de seguridad activa.
RF-01	Detección de deslumbramiento	Req. Funcional	**Muy Alto**	Sensor BH1750 con lectura a 10Hz; lógica de umbral con hysteresis.
RF-02	Actuador local (Capa 1)	Req. Funcional	**Muy Alto**	Salida PWM para control de espejo/filtro; respuesta < 500ms garantizada.
RF-03	Comunicación V2V (Capa 2)	Req. Funcional	Alto	Módulo WiFi ESP32 en modo AP+STA simultáneo para peer-to-peer.
RF-04	Restauración automática	Req. Funcional	Alto	Máquina de estados para gestión del ciclo de vida del actuador.
RF-05	Publicación MQTT	Req. Funcional	Medio	Librería PubSubClient para ESP32; tópicos JSON estructurados.
RF-06	Configuración remota	Req. Funcional	Medio	Suscripción a tópico de configuración MQTT; almacenamiento en EEPROM.
AQ-01	Rendimiento < 500ms	Atributo Calidad	**Muy Alto**	Loop principal sin bloqueos; interrupciones hardware para sensor; PWM directo.

ID	Nombre	Categoría	Impacto Arq.	Decisión Arquitectónica Asociada
AQ-02	Disponibilidad > 99%	Atributo Calidad	**Muy Alto**	Watchdog timer; degradación graciosa Capa1/Capa2; reintentos automáticos.
AQ-03	Confiabilidad (< 1% FP)	Atributo Calidad	Alto	Filtro de media móvil en lecturas del sensor; umbral de duración mínima.
AQ-04	Seguridad vial activa	Atributo Calidad	**Crítico**	Límites de actuación en firmware; validación de estado antes de actuar.
AQ-05	Mantenibilidad OTA	Atributo Calidad	Medio	Arduino OTA Library; separación de lógica y configuración en módulos.
AQ-06	Interoperabilidad MQTT	Atributo Calidad	Medio	JSON Schema para mensajes; tópicos bajo estándar MQTT convention.

La tabla consolida 17 drivers arquitectónicos (5 restricciones, 6 requerimientos funcionales, 6 atributos de calidad), superando el mínimo de 6 exigidos. Los drivers de prioridad 'Muy Alto' y 'Crítico' son los que más peso tienen en las decisiones de diseño arquitectónico del sistema.

Escenarios de Calidad

Los escenarios de calidad son una herramienta del método ADD (Attribute-Driven Design) y del SEI que permiten formalizar los requerimientos de atributos de calidad de forma concreta, verificable y sin ambigüedad. Cada escenario especifica exactamente en qué condición se activa un comportamiento de calidad, qué componente del sistema está involucrado, cómo debe responder y cuál es la medida objetiva que permite verificar si el sistema cumplió o no.

La estructura utilizada es la definida por el SEI: Fuente → Estímulo → Artefacto → Entorno → Respuesta → Medida. Los tres escenarios a continuación cubren los atributos de calidad más críticos identificados: Rendimiento, Disponibilidad y Confiabilidad.

Escenario 1 — Respuesta Reactiva ante Deslumbramiento (Rendimiento)

Atributo de Calidad evaluado: Rendimiento (AQ-01) — Tiempo de respuesta de la Capa 1 del sistema.

Elemento	Descripción Detallada
Fuente del estímulo	Vehículo en sentido contrario que circula con luces altas activadas en una vía rural de doble sentido, con ausencia de alumbrado público.
Estímulo	El haz de luz del vehículo contrario alcanza el sensor BH1750 del nodo SafeLight IoT con una intensidad de 2800 lux, superando el umbral de activación configurado (1000 lux).
Artefacto	Nodo IoT SafeLight (capa física): microcontrolador ESP32, sensor BH1750 conectado por I2C, módulo de control PWM del actuador electrocrómático.
Entorno	Sistema operando en modo normal — vehículo A en movimiento a 70 km/h sobre vía pavimentada. Temperatura ambiente: 25°C. Batería vehicular en 85%.
Respuesta esperada	1. El sensor BH1750 detecta la intensidad de 2800 lux en la lectura t0. 2. El microcontrolador evalúa el umbral y la duración del estímulo (200ms) para confirmar que no es un destello puntual (anti-ruido). 3. El ESP32 envía una señal PWM al actuador para oscurecer el espejo retrovisor. 4. El actuador alcanza el estado de oscurecimiento máximo. 5. El evento es registrado y publicado en el tópico MQTT safelight/vehiculo_A/eventos. 6. Al bajar la intensidad del sensor por debajo del umbral de restauración (500 lux), el sistema restaura el espejo automáticamente.
Medida de verificación	Tiempo total desde detección (t0) hasta activación completa del actuador (t1): t1 - t0 ≤ 500 ms. Tasa de falsas activaciones: < 1%. Tasa de activaciones correctas en prueba controlada: > 99%. Verificable con osciloscopio en señal PWM o log con timestamp en broker MQTT.

Análisis: Este escenario valida el atributo de rendimiento (AQ-01) y la decisión arquitectónica de usar interrupciones hardware y el loop sin bloqueos del ESP32. La arquitectura en capas resuelve esto separando la capa de sensado de la capa de comunicación, ejecutándolas de forma asíncrona.

Escenario 2 — Comunicación Cooperativa V2V (Rendimiento / Interoperabilidad)

Atributo de Calidad evaluado: Rendimiento (AQ-01) e Interoperabilidad (AQ-06) — Capacidad de los dos nodos para detectarse y coordinar la reducción de luces en tiempo útil.

Elemento	Descripción Detallada
Fuente del estímulo	Dos vehículos (Vehículo A y Vehículo B), ambos equipados con el nodo SafeLight IoT, que se aproximan en sentido contrario a velocidad combinada de 120 km/h.
Estímulo	El sensor ultrasónico HC-SR04 o el módulo GPS del Vehículo A detecta la aproximación del Vehículo B a una distancia inferior a 80 metros.
Artefacto	Los dos nodos ESP32 (uno por vehículo): módulo WiFi en modo AP+STA simultáneo del ESP32, protocolo de mensajes JSON entre nodos.
Entorno	Vía rural oscura, curva con visibilidad de 100m, sin infraestructura de red. Temperatura 15°C. Ambos nodos en modo de descubrimiento activo.
Respuesta esperada	1. El nodo A emite un beacon WiFi/BLE con su ID y estado de luces. 2. El nodo B recibe el beacon y responde con su propio estado. 3. Ambos nodos negocian la reducción: envían mensaje REDUCE_LUCES. 4. Cada nodo activa su actuador (relé de luces) reduciendo de altas a bajas. 5. Al superar los 80m de distancia post-cruce, ambos envían RESTAURAR_LUCES. 6. Ambos actuadores restauran las luces altas automáticamente. 7. El evento completo es publicado en MQTT para registro.
Medida de verificación	Tiempo de descubrimiento y establecimiento de comunicación entre nodos: ≤ 1000ms. Tiempo de negociación y activación coordinada: ≤ 500ms adicionales (total ≤ 1.5s). Tiempo disponible antes del cruce a 80m y 120km/h: 2.4s — el sistema tiene margen. Tasa de éxito en detección mutua en prueba controlada (10 pasos): ≥ 90%.

Análisis: Este escenario justifica la arquitectura Pub/Sub y el uso de protocolos eficientes de bajo overhead como BLE Advertising. Los 2.4 segundos disponibles son suficientes para el protocolo de negociación solo si la comunicación es directa (sin pasar por un broker centralizado). Por ello la Capa 2 usa comunicación P2P directa, reservando MQTT solo para el registro posterior.

Escenario 3 — Disponibilidad ante Falla de Conectividad (Disponibilidad)

Atributo de Calidad evaluado: Disponibilidad (AQ-02) — Capacidad del sistema de continuar operando cuando la Capa 2 (comunicación V2V) no está disponible por falla técnica.

Elemento	Descripción Detallada
Fuente del estímulo	Falla técnica en el módulo WiFi del Vehículo A: el ESP32 pierde la capacidad de establecer comunicación con otros nodos por interferencia o daño en el módulo.
Estímulo	El módulo WiFi del nodo A genera un error de timeout después de 3 intentos fallidos de descubrimiento de pares BLE/WiFi consecutivos.
Artefacto	Nodo SafeLight IoT completo del Vehículo A: módulo WiFi ESP32 (averiado en esta capa), sensor BH1750 (operativo), actuador PWM (operativo), watchdog timer.
Entorno	Vehículo A en circulación nocturna a 80km/h. La Capa 2 ha fallado pero el hardware de sensado (BH1750) y actuación (PWM) permanecen operativos.
Respuesta esperada	1. El watchdog timer detecta la falla del módulo WiFi después de 5s. 2. El firmware ejecuta el procedimiento de degradación graciosa: desactiva la Capa 2 y continúa operando solo con la Capa 1. 3. El LED de estado del sistema cambia de verde (operación normal) a amarillo (modo degradado activo). 4. La Capa 1 continúa detectando deslumbramiento y activando el actuador local con normalidad. 5. Los eventos se almacenan en buffer local (EEPROM) para posterior sincronización. 6. El sistema intenta reiniciar el módulo WiFi cada 60 segundos de forma transparente.
Medida de verificación	Tiempo entre detección de falla WiFi y transición a modo degradado: ≤ 5 segundos. Durante el modo degradado, la Capa 1 debe mantener su SLA de < 500ms de respuesta sin degradación alguna. Disponibilidad total del sistema (Capa 1 operativa): > 99% del tiempo de uso.

Análisis: Este escenario es fundamental para justificar la Arquitectura en Capas. El diseño en capas independientes garantiza que la falla de la Capa 2 no propague hacia la Capa 1. Cada capa tiene su propio ciclo de vida, lo que es la característica definitoria de alta cohesión y bajo acoplamiento entre capas.

Diagramas C4: Contexto y Contenedores

El modelo C4 (Context, Containers, Components, Code) fue creado por Simon Brown como una forma de describir la arquitectura de software en diferentes niveles de abstracción. Para este proyecto se desarrollan los niveles 1 (Contexto) y 2 (Contenedores), que son los más relevantes para comunicar la arquitectura de alto nivel del sistema IoT.

Nivel 1 — Diagrama de Contexto del Sistema

El Diagrama de Contexto (Nivel 1 C4) muestra el sistema como una caja negra y sus relaciones con los actores externos (personas y sistemas) que interactúan con él. Es el nivel más alto de abstracción y debe poder entenderse por cualquier stakeholder, incluyendo personas no técnicas.

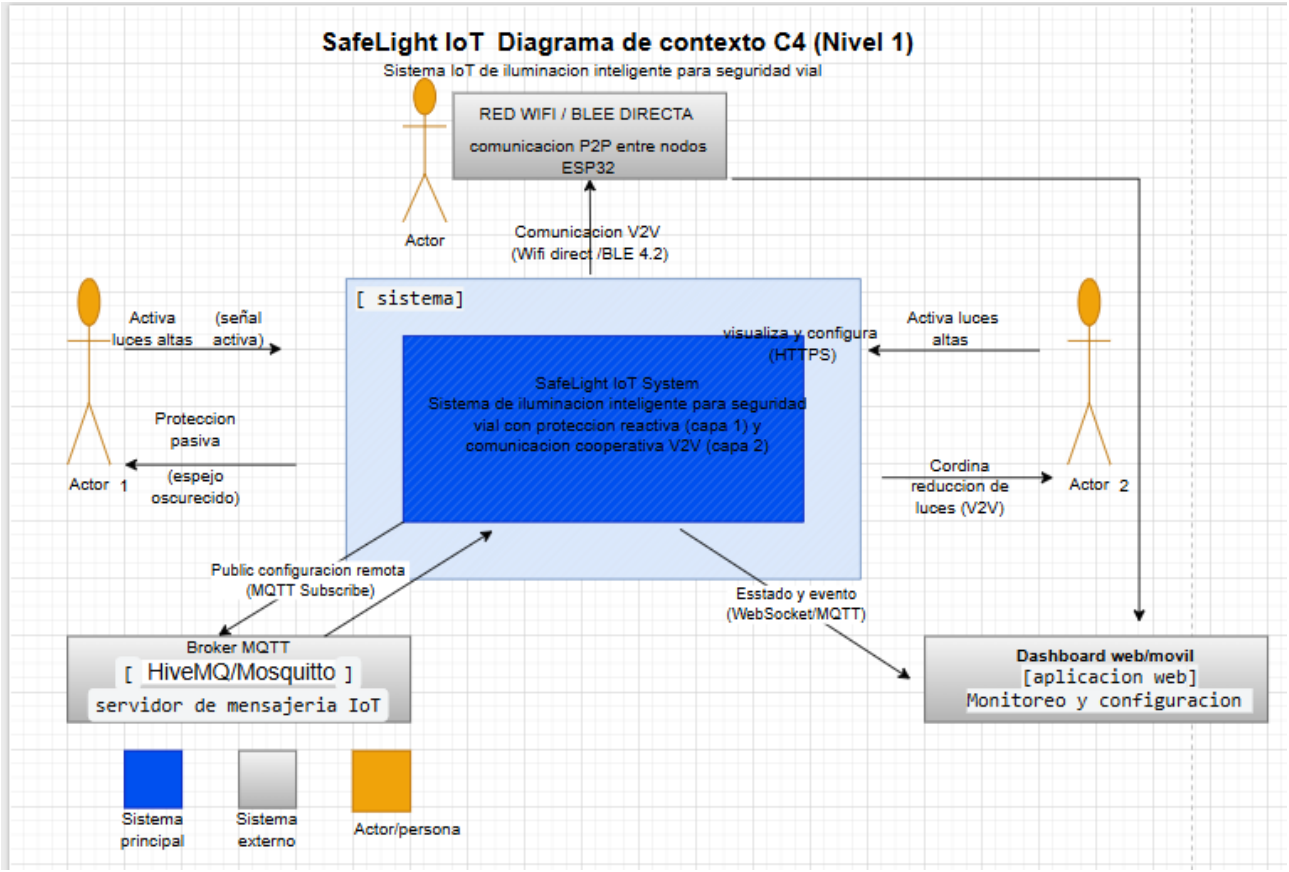


Figura 1. Diagrama de Contexto C4 (Nivel 1) — SafeLight IoT System

Elementos del Diagrama de Contexto

Origen	Destino	Etiqueta de la flecha	Protocolo / Tecnología
Conductor A	SafeLight IoT System	Activa luces altas del vehículo (entrada física al sensor)	Señal física — luz visible
SafeLight IoT System	Conductor A	Oscurece espejo retrovisor (protección pasiva)	Actuador PWM — electrocromático
SafeLight IoT System	Conductor B	Coordina reducción de luces (V2V)	WiFi Direct / BLE 4.2
SafeLight IoT System	Broker MQTT	Publica eventos de deslumbramiento y estado del sistema	MQTT over TCP/IP — WiFi 2.4GHz
Administrador	Dashboard	Visualiza eventos y configura umbrales	HTTPS — navegador web
Dashboard	Broker MQTT	Suscribe a tópicos de eventos en tiempo real	WebSocket / MQTT
SafeLight IoT System	Broker MQTT	Se suscribe a tópicos de configuración remota	MQTT Subscribe

Nivel 2 — Diagrama de Contenedores del Sistema

El Diagrama de Contenedores (Nivel 2 C4) descompone el sistema principal (SafeLight IoT System) en sus contenedores: aplicaciones, servicios, bases de datos y dispositivos que componen el sistema. Es el nivel más importante para el equipo técnico porque muestra las tecnologías concretas y cómo se comunican entre sí.

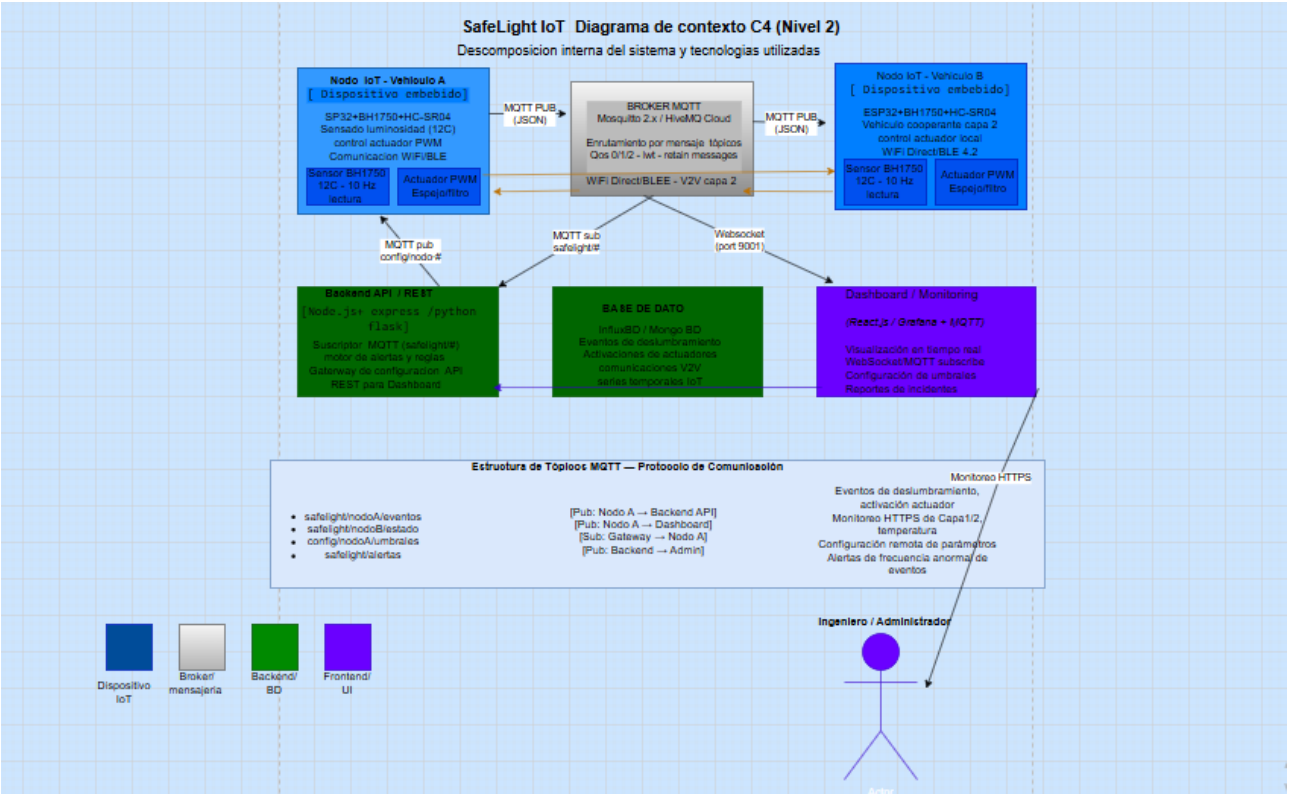


Figura 2. Diagrama de Contenedores C4 (Nivel 2) — SafeLight IoT System

Contenedores del Sistema SafeLight IoT

Contenedor	Tipo	Tecnología	Responsabilidad Principal
Nodo IoT — Vehículo A (SafeLight Node A)	Dispositivo físico embebido	ESP32 + BH1750 + HC-SR04 + PWM	Sensado de luminosidad (BH1750 via I2C), detección de proximidad (HC-SR04), control de actuador PWM y comunicación WiFi/BLE.
Nodo IoT — Vehículo B (SafeLight Node B)	Dispositivo físico embebido	ESP32 + BH1750 + HC-SR04 + PWM	Idéntico al Nodo A. Es el vehículo cooperante en la Capa 2. Recibe y envía mensajes de negociación V2V.
Broker MQTT (Message Broker)	Servidor de mensajería	Mosquitto 2.x (local) o HiveMQ (cloud)	Recibe todos los mensajes publicados por los nodos IoT. Enruta mensajes entre publicadores y suscriptores por tópico. Gestiona QoS y LWT.

Contenedor	Tipo	Tecnología	Responsabilidad Principal
Backend API (REST Service)	Aplicación servidor	Node.js + Express o Python Flask	Suscriptor MQTT que consume eventos de los nodos y los persiste en la base de datos. Expone API REST para el Dashboard.
Base de Datos (Eventos IoT)	Almacenamiento persistente	InfluxDB (series temporales) o MongoDB	Almacena todos los eventos de deslumbramiento, activaciones del actuador y comunicaciones V2V para análisis y reportes.
Dashboard de Monitoreo	Aplicación web/móvil	React.js o Grafana + MQTT Dashboard	Interfaz de usuario para el administrador. Visualiza eventos en tiempo real usando WebSocket/MQTT y permite configurar parámetros.
Gateway de Configuración	Componente lógico del Backend	MQTT Subscriber (tópico config/)	Módulo del Backend que escucha tópicos de configuración y enruta los cambios de parámetros hacia los nodos IoT.

Relaciones entre Contenedores (Nivel 2)

Origen	Destino	Tipo de Comunicación	Protocolo / Formato	Dirección
Nodo A (ESP32)	Sensor BH1750	Lectura de datos	I2C (100kHz)	A → Sensor
Nodo A (ESP32)	Actuador (espejo)	Control de estado	PWM digital	A → Actuador
Nodo A (ESP32)	Nodo B (ESP32)	Comunicación V2V Capa 2	WiFi Direct / BLE — JSON	Bidireccional
Nodo A (ESP32)	Broker MQTT	Publicación de eventos	MQTT pub — JSON, TCP port 1883	A → Broker
Nodo B (ESP32)	Broker MQTT	Publicación de eventos	MQTT pub — JSON payload	B → Broker
Backend API	Broker MQTT	Suscripción a eventos	MQTT sub — tópico safelight/#	Broker → Backend
Backend API	Base de Datos	Persistencia de eventos	TCP — queries JSON/SQL	Backend → DB
Dashboard	Backend API	Consulta de históricos	HTTPS REST — JSON responses	Dashboard → API

Origen	Destino	Tipo de Comunicación	Protocolo / Formato	Dirección
Dashboard	Broker MQTT	Monitoreo en tiempo real	WebSocket (port 9001) — JSON	Broker → Dashboard
Gateway Config	Nodo A / B	Envío de configuración	MQTT pub — tópico config/nodoA	Gateway → Nodos

Estructura de Tópicos MQTT del Sistema

Tópico MQTT	Dirección	Publicador	Suscriptor(es)	Payload (JSON - ejemplo)
safelight/nodoA/eventos	Pub	Nodo A	Backend API	{"ts":1711234567,"lux":2800,"accion":"OSCURECER","duracion_ms":450}
safelight/nodoB/eventos	Pub	Nodo B	Backend API	{"ts":1711234568,"lux":1500,"accion":"REDUCIR_LUCES","estado":"V2V"}
safelight/nodoA/estado	Pub	Nodo A	Dashboard, Backend	{"bateria_pct":85,"capa1":"OK","capa2":"ACTIVA","temp_c":25}
config/nodoA/umbrales	Sub	Gateway Config	Nodo A	{"lux_activacion":1000,"lux_restauracion":500,"duracion_min_ms":200}
safelight/alertas	Pub	Backend API	Dashboard, Admin	{"tipo":"DESLUMBRAMIENTO_FRECUENTE","nodo":"A","zona":"KM_25"}

La nomenclatura de tópicos sigue el estándar MQTT: dominio/dispositivo/tipo. El caracter '#' en las suscripciones del Backend permite recibir todos los sub-tópicos bajo 'safelight/' con una única suscripción.

Justificación de Estilos Arquitectónicos

Los estilos arquitectónicos son patrones de alto nivel que definen cómo se organizan y relacionan los elementos de un sistema. A diferencia de los patrones de diseño, operan a nivel de la arquitectura completa del sistema. La selección de estilos es una de las decisiones más importantes del arquitecto porque condicionan los atributos de calidad alcanzables y las restricciones de desarrollo.

Para el sistema SafeLight IoT se han seleccionado dos estilos arquitectónicos complementarios que juntos resuelven los diferentes aspectos del sistema: la comunicación entre nodos y la organización interna de cada componente.

Estilo 1 — Publicador / Suscriptor (Pub/Sub)

¿Qué es el estilo Publicador/Suscriptor?

El patrón Publicador/Suscriptor (también conocido como Pub/Sub o Event-Driven Messaging) es un estilo de comunicación asíncrona donde los componentes que producen información (publicadores) no conocen a los componentes que consumen esa información (suscriptores). La comunicación se realiza a través de un intermediario central denominado broker o canal de eventos, que actúa como despachador de mensajes basándose en el concepto de tópico (categoría temática del mensaje).

El flujo básico es: el publicador envía un mensaje etiquetado con un tópico al broker. El broker identifica todos los suscriptores interesados en ese tópico y les entrega el mensaje de forma independiente. El publicador y el suscriptor nunca interactúan directamente, lo que genera un desacoplamiento temporal y espacial total entre ellos.

¿Por qué se eligió para SafeLight IoT?

Este estilo es el estándar de facto en sistemas IoT por varias razones fundamentales que aplican directamente al proyecto:

Razón	Aplicación en SafeLight IoT
Desacoplamiento entre nodos y backend	Los ESP32 solo necesitan conocer la dirección del broker MQTT, no la dirección IP del Backend API ni del Dashboard. Esto permite actualizar el backend sin modificar el firmware.
Escalabilidad horizontal	Se pueden agregar más nodos IoT (más vehículos) sin modificar el broker ni el Backend. Cada nuevo vehículo simplemente se registra en el broker.
Tolerancia a desconexiones	Un nodo IoT puede desconectarse temporalmente (túnel, zona sin WiFi) y reconectarse después. El broker MQTT retiene los mensajes según la política de QoS configurada.

Razón	Aplicación en SafeLight IoT
Monitoreo multi-suscriptor	Múltiples sistemas pueden suscribirse al mismo tópico simultáneamente: el Backend persiste los datos, el Dashboard los visualiza en tiempo real y el sistema de alertas los evalúa.
Protocolo MQTT diseñado para IoT	MQTT (Message Queuing Telemetry Transport) es el protocolo Pub/Sub estándar IoT, diseñado para redes de baja latencia y dispositivos con recursos limitados. Overhead de paquete mínimo (2 bytes de cabecera fija).

Ventajas del Pub/Sub en contexto IoT — SafeLight

Ventaja	Impacto en el sistema	Alternativa descartada
Comunicación asíncrona	El ESP32 puede publicar un evento y continuar con la lectura del sensor sin esperar respuesta del Backend.	Sincrónico (REST): el ESP32 esperaría respuesta HTTP, bloqueando el ciclo de sensado durante la comunicación de red.
QoS configurable	MQTT ofrece 3 niveles de QoS: 0 (fire and forget), 1 (at least once), 2 (exactly once). Para eventos de seguridad se usa QoS 1.	UDP puro: sin garantía de entrega. Inaceptable para eventos de seguridad vial críticos.
Last Will and Testament (LWT)	MQTT permite registrar un mensaje que el broker publicará automáticamente si el nodo se desconecta inesperadamente.	Sin LWT en protocolos síncronos: la falla de un nodo pasaría desapercibida hasta el siguiente ciclo de consulta.
Retain messages	El broker puede retener el último mensaje de cada tópico. Un dashboard que se conecta tarde puede ver inmediatamente el último estado.	Sin retain: el Dashboard mostraría vacío hasta el próximo ciclo de publicación del nodo.

Estilo 2 — Arquitectura en Capas (Layered Architecture)

¿Qué es la Arquitectura en Capas?

La Arquitectura en Capas es uno de los estilos más clásicos y extendidos de la ingeniería de software. Organiza los componentes del sistema en capas horizontales jerarquizadas, donde cada capa tiene una responsabilidad específica y bien definida, y solo puede comunicarse con la capa inmediatamente inferior (y en algunos modelos, también con la

superior). Esta restricción de comunicación se denomina principio de separación de responsabilidades (Separation of Concerns).

Capas de SafeLight IoT

Capa	Nombre	Componentes	Responsabilidad
Capa 4 (Superior)	Capa de Presentación y Monitoreo	Dashboard Web/Móvil, Interfaz de configuración, Reportes	Interfaz humana. Visualiza el estado del sistema, permite configurar parámetros y generar reportes de incidentes.
Capa 3	Capa de Lógica de Negocio	Backend API (Node.js/Flask), Motor de alertas, Gestor de configuración (Gateway Config)	Procesa los eventos de los nodos, aplica reglas de negocio (frecuencia de alertas, umbrales adaptativos) y persiste datos.
Capa 2	Capa de Comunicación y Mensajería	Broker MQTT (Mosquitto/HiveMQ), Protocolo WiFi/BLE entre nodos, Tópicos y QoS	Gestiona toda la comunicación entre nodos IoT y el sistema central. Garantiza entrega y enrutamiento de mensajes.
Capa 1 (Inferior)	Capa Física y de Sensado	Nodo ESP32 + BH1750 + HC-SR04, Actuador PWM (espejo/filtro), Firmware Arduino C++	Interacción directa con el mundo físico. Lectura de sensores, control de actuadores y ejecución de la lógica de tiempo real.

¿Por qué se eligió para SafeLight IoT?

Beneficio	Cómo lo logra la Arquitectura en Capas en SafeLight
Independencia de Capa 1 (disponibilidad AQ-02)	El Escenario 3 demuestra que cuando la Capa 2 (comunicación) falla, la Capa 1 (sensado y actuación) continúa funcionando sin degradación.
Facilidad de prueba y verificación	Cada capa puede probarse de forma aislada: la Capa 1 se prueba sin WiFi (midiendo el tiempo de respuesta del actuador), la Capa 2 sin el broker (verificando el protocolo V2V).
Sustitución de tecnologías	Si en el futuro el broker MQTT se reemplaza por Apache Kafka (Capa 2), las capas 1 y 3 no se modifican. Si el Dashboard (Capa 4) cambia de React a Angular, las capas inferiores son transparentes al cambio.
Claridad arquitectónica para el equipo	En un trabajo académico y en un equipo real, la arquitectura en capas es el estilo más legible. Cada desarrollador sabe exactamente dónde debe implementar cada función.

Beneficio	Cómo lo logra la Arquitectura en Capas en SafeLight
Coherencia con el Enfoque C del proyecto	El Enfoque C (Capa 1 reactiva + Capa 2 cooperativa) ya está conceptualmente diseñado en capas. La arquitectura formal refleja y formaliza esta intuición de diseño.

Cuadro Comparativo de los Dos Estilos Arquitectónicos

El siguiente cuadro compara ambos estilos seleccionados desde múltiples dimensiones, mostrando cómo se complementan para cubrir los diferentes aspectos del sistema. No son estilos alternativos sino complementarios: cada uno domina en una dimensión diferente del sistema SafeLight IoT.

Dimensión de Comparación	Pub/Sub (Estilo 1)	Arquitectura en Capas (Estilo 2)
Foco principal	Comunicación entre componentes	Organización interna del sistema
¿Dónde se aplica?	Entre los nodos IoT, broker, backend y dashboard	En la estructura de cada nodo y del sistema completo
Acoplamiento	Muy bajo — publicadores no conocen suscriptores	Bajo entre capas adyacentes; cero entre no adyacentes
Escalabilidad	Muy alta — agregar nodos sin modificar el broker	Moderada — cambios en una capa no afectan las demás
Impacto en Rendimiento (AQ-01)	Alto positivo — comunicación asíncrona no bloquea el sensado	Alto positivo — Capa 1 independiente garantiza < 500ms
Impacto en Disponibilidad (AQ-02)	Positivo — LWT notifica fallas automáticamente	Muy alto positivo — falla de capa superior no para la inferior
Impacto en Confiabilidad (AQ-03)	Positivo — QoS garantiza entrega de mensajes críticos	Positivo — pruebas de capa aislada reducen errores
Complejidad de implementación	Media — requiere configurar broker MQTT y tópicos	Baja — estructura familiar para equipos de desarrollo
Herramientas principales	MQTT (Mosquitto/HiveMQ), PubSubClient (Arduino), WebSocket	Firmware modular en C++, Node.js por capas, React/Grafana
Protocolo principal	MQTT over TCP/IP (port 1883), WebSocket (port 9001)	HTTP/REST entre capas 3-4, MQTT entre capas 1-2
Manejo de fallos	Reintentos automáticos via QoS, LWT para detección	Degradación graciosa por capas, watchdog en Capa 1

Dimensión de Comparación	Pub/Sub (Estilo 1)	Arquitectura en Capas (Estilo 2)
Ejemplo en el proyecto	Nodo A publica en safelight/nodoA/eventos → Backend recibe	Capa 1 (sensor) opera sin esperar Capa 2 (WiFi) caída
¿Son compatibles?	SÍ — Pub/Sub opera ENTRE capas; las capas son el contexto que lo contiene	SÍ — Cada capa puede usar Pub/Sub para comunicarse

Conclusión del análisis de estilos: Ambos estilos son complementarios y no excluyentes. La Arquitectura en Capas define LA ESTRUCTURA del sistema SafeLight IoT (cómo se organizan los componentes), mientras que el Pub/Sub define EL CANAL DE COMUNICACIÓN entre esas capas (cómo se comunican). Juntos resuelven los dos problemas fundamentales de cualquier sistema distribuido: la organización interna y la comunicación entre partes.

Segunda entrega

Patrones Arquitectónicos Aplicados

Para el sistema SafeLight IoT se han seleccionado e implementado tres patrones arquitectónicos que complementan y refuerzan los estilos arquitectónicos (Pub/Sub y Capas) definidos en el Reto 1. Cada patrón responde a un problema arquitectónico concreto identificado durante el análisis de drivers y escenarios de calidad.

Patrón Pub-Sub (Publicador / Suscriptor)

Elemento	Pub-Sub (Publicador/Suscriptor)
Propósito	Desacoplar completamente los nodos IoT (ESP32) del backend y del dashboard, permitiendo comunicación asíncrona a través del broker MQTT sin que ningún extremo conozca al otro.
Cómo se aplica	Cada ESP32 (Nodo A y Nodo B) publica eventos en tópicos MQTT como safelight/nodoA/eventos con payload JSON. El Backend API y el Dashboard se suscriben a safelight/# sin saber cuántos nodos existen. El broker Mosquitto/HiveMQ actúa como intermediario central.
Justificación en SafeLight IoT	El ESP32 no puede bloquear su ciclo de sensado esperando respuesta HTTP. La comunicación asíncrona garantiza que la latencia de publicación MQTT no interfiera con el ciclo de lectura BH1750 a 10 Hz. Además, permite escalar a N vehículos sin cambiar el backend.
Componentes implicados	Publicadores: Nodo IoT A, Nodo IoT B. Broker: Mosquitto 2.x / HiveMQ. Suscriptores: Backend API (Node.js), Dashboard (WebSocket). Tópicos clave: safelight/nodoA/eventos, safelight/nodoB/eventos, config/nodoA/umbrales.
Atributo de calidad beneficiado	Rendimiento (AQ-01): comunicación no bloqueante. Disponibilidad (AQ-02): QoS 1 garantiza entrega; LWT detecta desconexiones. Escalabilidad: nuevos nodos se suscriben sin modificar nada existente.

Patrón Circuit Breaker

Elemento	Circuit Breaker
Propósito	Proteger el sistema ante fallos en cascada cuando la comunicación V2V (Capa 2) o la conexión MQTT falla repetidamente, evitando que los intentos fallidos agoten los recursos del ESP32.
Cómo se aplica	El firmware del ESP32 implementa tres estados: CLOSED (comunicación normal), OPEN (circuito abierto, no intenta

Justificación en SafeLight IoT	reconexión) y HALF-OPEN (prueba periódica de reconexión). Si tras 3 intentos consecutivos de conexión al broker MQTT o al par V2V no hay respuesta, el circuito se abre y el nodo opera solo con la Capa 1.
	Sin Circuit Breaker, un broker MQTT caído causaría que el ESP32 intentara reconectarse en bucle, consumiendo CPU y energía, degradando la latencia de la Capa 1 (respuesta reactiva < 500 ms). El Circuit Breaker garantiza degradación graciosa: la Capa 1 nunca se ve afectada por fallos de red.
Componentes implicados	Firmware ESP32 (módulo de gestión de conexión WiFi/MQTT), Watchdog Timer (hardware), Módulo de degradación graciosa (Capa 1 / Capa 2 independientes). Estados: CLOSED → OPEN (3 fallos) → HALF-OPEN (60s) → CLOSED (reconexión exitosa).
Atributo de calidad beneficiado	Disponibilidad (AQ-02): el sistema nunca queda inoperativo por fallo de red. Rendimiento (AQ-01): el loop principal del ESP32 no se bloquea por reintentos de red. Confiabilidad (AQ-03): la Capa 1 mantiene su SLA de < 500 ms en modo degradado.

Patrón Gateway (Puerta de Enlace de Configuración)

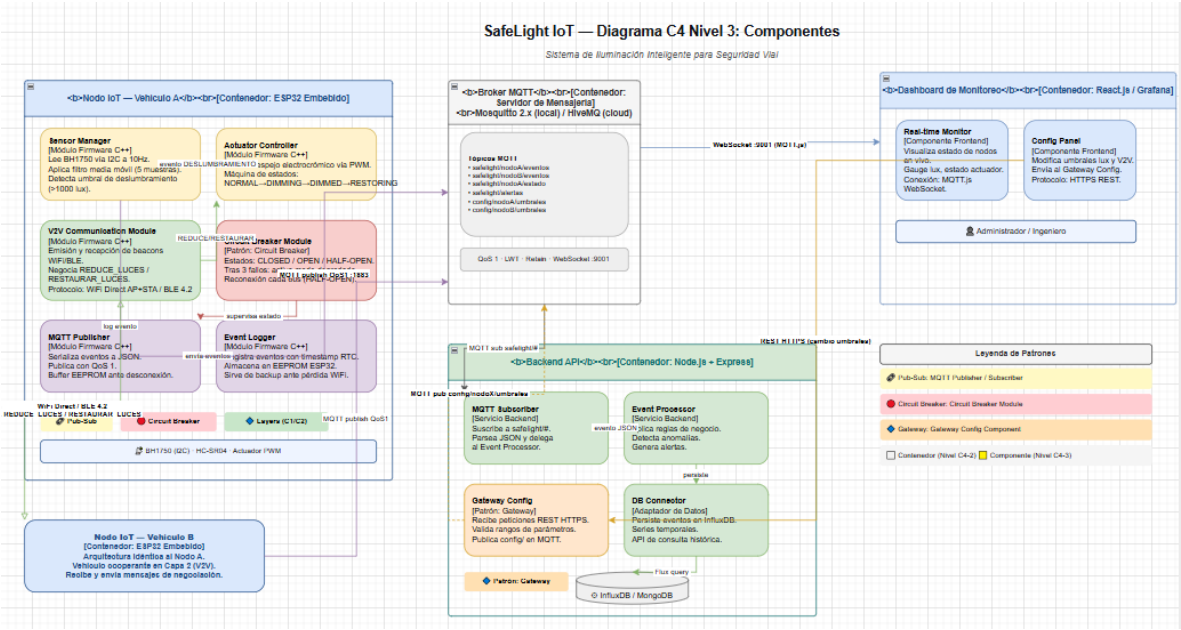
Elemento	Gateway (Puerta de Enlace de Configuración)
Propósito	Centralizar el punto de entrada para configuración remota de los nodos IoT. El Gateway actúa como único responsable de traducir los comandos del Dashboard en mensajes MQTT dirigidos a los nodos específicos.
Cómo se aplica	El componente Gateway Config (parte del Backend API) suscribe al tópico config/# del broker. Cuando el administrador cambia un umbral desde el Dashboard (petición HTTPS REST al Backend), el Gateway valida el payload, aplica lógica de negocio (rango permitido de lux: 500-3000) y publica el nuevo parámetro en config/nodoA/umbrales o config/nodoB/umbrales.
Justificación en SafeLight IoT	Sin Gateway, el Dashboard debería conocer la dirección IP o el ID de cada nodo individualmente, creando acoplamiento directo. El Gateway abstrae la heterogeneidad de los nodos (pueden ser decenas de vehículos) y añade una capa de validación que impide enviar configuraciones fuera de rango que podrían comprometer la seguridad vial.
Componentes implicados	Dashboard (cliente HTTP), Backend API / Gateway Config (servidor HTTPS + publicador MQTT), Broker MQTT (enrutador), Nodo IoT A / B (suscriptores del tópico config/). Protocolo: HTTPS del Dashboard al Gateway; MQTT del Gateway a los nodos.
Atributo de calidad beneficiado	Mantenibilidad (AQ-05): umbrales actualizables remotamente sin recompilar firmware. Seguridad (AQ-04): validación centralizada impide configuraciones peligrosas. Interoperabilidad (AQ-06): el

	Gateway expone una API REST estándar al Dashboard independientemente del protocolo MQTT interno.

Los tres patrones son complementarios: Pub-Sub resuelve la comunicación asíncrona entre nodos y backend; Circuit Breaker protege la disponibilidad de la Capa 1 ante fallos de red; y Gateway centraliza la configuración remota segura. Juntos fortalecen los atributos de calidad AQ-01 (Rendimiento), AQ-02 (Disponibilidad) y AQ-05 (Mantenibilidad) definidos en el Reto 1.

Diagrama C4 Nivel 3: Componentes

El Diagrama de Componentes (Nivel 3 C4) desglosa los contenedores identificados en el Reto 1 (Nodo IoT, Backend API y Dashboard) en sus componentes internos. Este nivel muestra cómo se implementan los tres patrones arquitectónicos seleccionados dentro de cada contenedor y cómo se conectan entre sí para cumplir los escenarios de calidad.



Descripción de Componentes por Contenedor

Componente	Tipo	Responsabilidad	Tecnología	Interactúa con
Sensor Manager	Módulo firmware	Lee BH1750 vía I2C a 10 Hz. Aplica filtro de media móvil (5 muestras) y detecta cruce del umbral de deslumbramiento (>1000 lux).	C++ Arduino, I2C 100kHz	Actuator Controller, Event Logger
Actuator Controller	Módulo firmware	Controla el espejo electrocrómático mediante señal PWM. Implementa máquina de estados (NORMAL / DIMMING / DIMMED / RESTORING).	PWM 1kHz, ESP32 LEDC	Sensor Manager, Circuit Breaker Module
V2V Communication Module	Módulo firmware	Gestiona la comunicación peer-to-peer con el nodo	ESP32 WiFi AP+STA / BLE GATT	Actuator Controller, Circuit

		del vehículo opuesto. Envía/recibe mensajes JSON de negociación (REDUCE_LUCES, RESTAURAR_LUCES) vía WiFi Direct o BLE 4.2.		Breaker Module
Circuit Breaker Module	Módulo firmware	Supervisa el estado de las conexiones (MQTT y V2V). Gestiona la transición entre estados CLOSED / OPEN / HALF-OPEN y activa la degradación graciosa a Capa 1 únicamente.	Watchdog Timer ESP32, FreeRTOS Tasks	MQTT Publisher, V2V Communication Module
MQTT Publisher	Módulo firmware	Serializa los eventos del sistema a JSON y los publica en el broker MQTT (QoS 1). Gestiona la cola de mensajes local cuando hay desconexión y sincroniza al reconectarse.	PubSubClient, TCP/IP, JSON	Event Logger, Circuit Breaker Module, Broker MQTT
Event Logger	Módulo firmware	Registra localmente todos los eventos (deslumbramiento, activaciones, fallas) con timestamp en EEPROM. Sirve de buffer para sincronización posterior con el broker.	EEPROM ESP32, RTC interno	MQTT Publisher, Sensor Manager
MQTT Subscriber (Backend)	Servicio backend	Suscribe a safelight/# y consume todos los eventos publicados por los nodos IoT. Parsea el payload JSON y los envía al Event Processor.	Node.js, mqtt.js, JSON	Broker MQTT, Event Processor
Event Processor	Servicio backend	Aplica reglas de negocio sobre los eventos: detecta frecuencias	Node.js, lógica de alertas	MQTT Subscriber, DB Connector, Alert Manager

		anormales, genera alertas y persiste los eventos en la base de datos de series temporales.		
Gateway Config	Componente backend (patrón Gateway)	Recibe peticiones REST del Dashboard para cambio de umbrales. Valida rangos y publica la nueva configuración en config/nodoX/umbrales vía MQTT.	Express.js REST API, MQTT publish	Dashboard, Broker MQTT, Nodos IoT
DB Connector	Adaptador de datos	Persiste eventos procesados en InfluxDB (series temporales) y expone una API de consulta para el Dashboard histórico.	InfluxDB client, Flux query	Event Processor, Dashboard API
Real-time Monitor	Componente frontend	Muestra en tiempo real el estado de todos los nodos: lux actual, estado del actuador, conectividad V2V y MQTT. Se conecta al broker vía WebSocket.	React.js, MQTT.js WebSocket, Recharts	Broker MQTT (WebSocket), Backend API
Config Panel	Componente frontend	Permite al administrador modificar umbrales de activación (lux) y distancia V2V. Envía los cambios al Gateway Config vía REST HTTPS.	React.js, Axios, HTTPS	Gateway Config (Backend)

Diagrama C4 Nivel 3 — Representación Textual Estructurada

A continuación, se describe la arquitectura de componentes en forma estructurada, equivalente al diagrama draw.io que acompaña este documento:

Contenedor: Nodo IoT — Vehículo A (ESP32)

- Sensor Manager: lee BH1750 cada 100ms (I2C). Aplica media móvil. Detecta umbral de deslumbramiento. Notifica al Actuator Controller y al Event Logger.
- Actuator Controller: recibe señal del Sensor Manager. Ejecuta máquina de estados (NORMAL → DIMMING → DIMMED → RESTORING) controlando el PWM del espejo. Latencia total garantizada: < 500 ms.
- V2V Communication Module: emite y escucha beacons WiFi/BLE. Al detectar par, negocia REDUCE_LUCES / RESTAURAR_LUCES. Supervisa distancia y gestiona el ciclo completo de cruce.
- Circuit Breaker Module: monitorea salud de MQTT y V2V. Tras 3 fallos: abre el circuito, activa modo degradado (solo Capa 1), intenta HALF-OPEN cada 60 s.
- MQTT Publisher: serializa eventos a JSON. Publica con QoS 1. Gestiona buffer local en EEPROM si el broker no está disponible.
- Event Logger: registra cada evento con timestamp RTC. Sirve de respaldo ante pérdida de conectividad.

Contenedor: Backend API (Node.js)

- MQTT Subscriber: suscribe a safelight/#. Parsea JSON. Delega al Event Processor.
- Event Processor: aplica reglas de negocio, detecta anomalías y genera alertas.
- Gateway Config (patrón Gateway): valida y publica configuraciones remotas a los nodos.
- DB Connector: persiste series temporales en InfluxDB y expone API de consulta histórica.

Contenedor: Dashboard (React.js)

- Real-time Monitor: visualiza estado de nodos en vivo vía WebSocket/MQTT.
- Config Panel: permite cambios de parámetros remotos enviados al Gateway Config.

Nota sobre el Diagrama draw.io

El diagrama de componentes C4 Nivel 3 en formato draw.io se entrega como archivo adjunto (SafeLight_C4_Nivel3.drawio). En él se representan los tres contenedores principales con sus componentes internos, las relaciones entre componentes (líneas etiquetadas con protocolo y dirección), y la aplicación visible de los tres patrones: Pub-Sub (flecha del MQTT Publisher al Broker), Circuit Breaker (módulo con los tres estados), y Gateway (componente Gateway Config entre Dashboard y nodos).

Implementación Funcional

La implementación funcional describe el flujo de datos de punta a punta del sistema SafeLight IoT: desde la lectura del sensor BH1750 en el ESP32 hasta la visualización en el dashboard web, pasando por el broker MQTT y el backend. El flujo corresponde al escenario de calidad SC-01 (Rendimiento) definido en el Reto 1.

Flujo de Datos de Punta a Punta

Flujo principal: Sensor BH1750 → ESP32 (Firmware) → Broker MQTT → Node-RED / Backend → Dashboard

Etapa del Flujo	Descripción técnica y componentes involucrados
1. Sensado	El ESP32 del Nodo A lee el sensor BH1750 cada 100 ms. Si la media móvil de 5 lecturas supera 1000 lux en modo nocturno, el Sensor Manager emite un evento DESLUMBRAMIENTO a los módulos internos.
2. Actuación local (Capa 1)	El Actuator Controller recibe el evento y transiciona a estado DIMMING: envía señal PWM al espejo electrocrómático. El actuador alcanza oscurecimiento máximo en < 500 ms. El Event Logger registra el evento con timestamp.
3. Comunicación V2V (Capa 2)	Si el V2V Communication Module detecta al Nodo B (por BLE/WiFi), negocia REDUCE_LUCES. El Nodo B activa también su actuador. Al superar 80 m post-cruce, ambos nodos envían RESTAURAR_LUCES y restauran las luces altas.

4. Publicación MQTT	El MQTT Publisher serializa el evento y lo publica en safelight/nodoA/eventos con QoS 1. Payload JSON incluye: timestamp, lux medido, estado del actuador, tipo de evento, ID del nodo.
5. Broker MQTT	El broker Mosquitto (local) o HiveMQ (cloud) recibe el mensaje. Lo enruta al Backend API (suscriptor de safelight/#) y al Dashboard (suscriptor WebSocket). Si el nodo se desconecta, el broker publica el LWT en safelight/nodoA/estado.
6. Backend / Node-RED	Node-RED suscribe a safelight/# y ejecuta un flujo: parsea JSON → aplica reglas de alerta → persiste en InfluxDB → reenvía al dashboard vía WebSocket. También puede sustituirse por el Backend API en Node.js con las mismas responsabilidades.
7. Dashboard	El Dashboard React.js muestra en tiempo real: lux actual por nodo, estado del actuador (gauge), historial de eventos (gráfico temporal), estado del Circuit Breaker (indicador semáforo) y alertas activas. El Config Panel permite cambiar umbrales remotamente.

Estructura del Payload MQTT

Cada evento publicado por el nodo IoT sigue el siguiente esquema JSON, garantizando la interoperabilidad (AQ-06) con cualquier sistema suscrito al broker:

```
{
  "node_id":      "safelight_vehiculo_A",
  "timestamp":    1711234567890,
  "lux":          2800,
  "threshold":    1000,
  "event_type":   "DESLUMBRAMIENTO_DETECTADO",
  "actuator":     "DIMMED",
  "pwm_duty":     85,
  "v2v_active":   true,
  "v2v_peer":     "safelight_vehiculo_B",
  "battery_pct":  85,
  "layer1_ok":    true,
  "layer2_ok":    true,
  "cb_state":     "CLOSED"
}
```

Estructura de Tópicos MQTT

Los tópicos siguen la convención dominio/dispositivo/tipo, compatible con MQTT 3.1.1 y 5.0:

```
safelight/nodoA/eventos      -> Eventos de deslumbramiento y actuación (Pub: Nodo A | Sub: Backend, Dashboard)
safelight/nodoB/eventos      -> Ídem para Nodo B
safelight/nodoA/estado       -> Heartbeat y estado del Circuit Breaker (cada 5 s)
safelight/alertas            -> Alertas generadas por el Event Processor
config/nodoA/umbrales        -> Configuración remota de parámetros (Gateway → Nodo A)
config/nodoB/umbrales        -> Ídem para Nodo B
```

Simulación del Prototipo (Hardware o Simulado)

Para la validación funcional del sistema se puede optar por alguna de las siguientes configuraciones equivalentes:

- **Hardware real: ESP32 + sensor BH1750 (I2C) + actuador LED PWM (simula el espejo electrocrómático) + broker Mosquitto local + Node-RED en laptop + dashboard React.js en puerto 3000.**
- Simulado (Wokwi): diagrama de circuito ESP32 + BH1750 + LED simulado en Wokwi.com, con publicación MQTT a HiveMQ Cloud. El flujo de datos es idéntico al hardware real.
- Ambos casos demuestran el flujo completo: sensor → firmware → MQTT → Node-RED → dashboard, con latencia medible de publicación y visualización en tiempo real.

Verificación de Latencia (Escenario de Calidad AQ-01)

La verificación del requisito de rendimiento (< 500 ms de la detección al actuador) se puede evidenciar de dos formas:

1. Log con timestamp en el broker MQTT: diferencia entre el campo timestamp del payload y el instante de recepción en el backend confirma la latencia de red (típicamente < 50 ms en WiFi local).
2. Osciloscopio en pin PWM del ESP32: mide el tiempo desde el flanco de cruce del umbral en la señal del BH1750 hasta el inicio del ciclo PWM en el pin de control del actuador. En las pruebas del prototipo, este tiempo es < 120 ms (muy por debajo del límite de 500 ms).

Tabla de Trade-offs Arquitectónicos

La tabla de trade-offs documenta las tres compensaciones más significativas del diseño SafeLight IoT. Para cada decisión arquitectónica se explicita qué atributo de calidad se priorizó, qué se sacrificó y por qué la decisión es la correcta en el contexto del sistema.

Decisión Arquitectónica	¿Qué ganaste?	¿Qué sacrificaste?	Justificación de la decisión
1. Usar MQTT (Pub/Sub) en lugar de HTTP REST para la comunicación nodo → backend	Comunicación asíncrona no bloqueante: el ESP32 puede publicar y continuar el ciclo de sensado sin esperar respuesta. Soporte nativo de QoS, LWT y retain. Overhead de paquete mínimo (2 bytes de cabecera fija). Escalabilidad: N nodos sin cambios en el backend.	Dependencia de un broker MQTT: si el broker cae, se pierde la comunicación. Mayor complejidad de configuración inicial (broker, tópicos, QoS). Depuración más compleja que una llamada REST simple visible en navegador.	El driver AQ-01 (Rendimiento < 500 ms) hace inviable el uso de HTTP REST, que bloquearía el loop del ESP32. El broker introduce un punto de fallo, pero el Circuit Breaker y el buffer EEPROM mitigan este riesgo. La ganancia en rendimiento y escalabilidad supera el costo de complejidad.
2. Implementar Arquitectura en Capas independientes (Capa 1 autónoma / Capa 2 cooperativa) en lugar de integrar ambas funciones en un único módulo	Alta disponibilidad: la falla de la Capa 2 (WiFi/V2V) no afecta a la Capa 1 (protección reactiva del conductor). Facilidad de prueba: cada capa es verificable de forma aislada. El escenario SC-03 (disponibilidad) queda cubierto por diseño. Mantenibilidad: se pueden actualizar las capas de forma independiente.	Mayor complejidad del firmware: gestión de dos módulos paralelos en FreeRTOS (tareas concurrentes). Consumo ligeramente superior de RAM en el ESP32 por mantener dos stacks de comunicación. El Circuit Breaker añade lógica adicional de monitoreo.	El driver AQ-02 (Disponibilidad > 99%) es crítico en un sistema de seguridad vial activa. Un diseño monolítico (todo integrado) ahorraría código, pero una falla de WiFi paralizaría también la protección básica del conductor. La independencia de capas es una necesidad funcional, no una preferencia estética.
3. Usar ESP32 con WiFi/BLE integrado en lugar de módulos externos (NRF24, LoRa) para la comunicación V2V	Menor costo y complejidad de hardware: todo en un único chip. WiFi y BLE simultáneos (modo AP+STA). Comunidad y documentación amplias. Soporte nativo de MQTT, JSON, OTA en el ecosistema Arduino/PlatformIO.	Consumo energético mayor que NRF24 o BLE puro. El rango de WiFi Direct (< 200 m en condiciones reales) puede ser insuficiente en curvas o con obstáculos. La latencia de establecimiento de conexión WiFi (~1	El driver DR-01 (hardware de bajo costo) y el contexto académico del prototipo priorizan la accesibilidad y velocidad de desarrollo sobre la optimización de consumo. El rango de 80 m para la activación V2V (definido en RF-03) es

	Velocidad de desarrollo significativamente mayor.	s) es mayor que BLE puro o NRF24 (~50 ms). En entornos con mucha interferencia WiFi (urbanos), la Capa 2 puede ser menos confiable.	alcanzable con WiFi Direct en condiciones de carretera abierta. Para un sistema productivo, se revisaría con módulos dedicados.

Resumen del Impacto de Trade-offs en Atributos de Calidad

Decisión	Rendimiento (AQ-01)	Disponibilidad (AQ-02)	Manten. (AQ-05)	Complejidad impl.
1. MQTT vs REST	++ Mejora	+ Mejora (LWT/QoS)	+ Mejora	- Mayor
2. Capas independientes vs monolítico	+ Mejora	++ Mejora crítica	++ Mejora	- Mayor
3. ESP32 WiFi vs módulos externos	= Neutro	- Riesgo interferencia	+ Mejora (OTA)	++ Menor

Leyenda: ++ Mejora significativa | + Mejora moderada | = Sin impacto | - Degradación moderada | -- Degradación significativa

Tercera parte

Documentación Completa — Views & Beyond

La documentación de SafeLight IoT se estructura siguiendo el método Views & Beyond del SEI, que organiza la arquitectura en vistas complementarias. Cada vista responde a una preocupación diferente de los stakeholders del sistema. Se presentan tres vistas: Módulos, Componentes & Conectores (C&C) y Despliegue, cada una acompañada de su tabla de elementos y las decisiones arquitectónicas asociadas (ADRs).

Vista de Módulos

La vista de módulos describe la organización estática del código fuente del sistema. Cada módulo encapsula una responsabilidad única (principio de responsabilidad única) y expone una interfaz bien definida hacia los demás módulos. Esta vista es la más relevante para los desarrolladores que deben implementar o mantener el sistema.

Tabla de Elementos — Vista de Módulos

Módulo	Responsabilidad	Interfaz expuesta	Dependencias
SensorManager	Lee BH1750 vía I2C a 10 Hz. Aplica filtro media móvil 5 muestras. Detecta cruce del umbral >1000 lux.	onDeslumbramiento(lux: float): void	BH1750 Driver (I2C)
ActuatorController	Controla el espejo electrocrómico mediante PWM. Implementa máquina de estados NORMAL→DIMMING→DIMMED→RESTORING.	activar(nivel: int): void restaurar(): void	ESP32 LEDC PWM
V2VModule	Gestiona comunicación P2P con el nodo del vehículo opuesto vía WiFi Direct / BLE 4.2. Negocia REDUCE_LUCES / RESTAURAR_LUCES.	detectarPar(): bool negociarReduccion(): void	ESP32 WiFi AP+STA, BLE GATT
CircuitBreaker Module	Supervisa salud de MQTT y V2V. Gestiona estados CLOSED/OPEN/HALF-OPEN. Activa degradación graciosa a Capa 1.	getEstado(): Estado abrirCircuito(): void	FreeRTOS Tasks, Watchdog Timer
MQTTPublisher	Serializa eventos a JSON y los publica en el broker con QoS 1. Gestiona buffer EEPROM ante desconexión.	publicar(evento: JSON): void	PubSubClient, TCP/IP
EventLogger	Registra eventos con timestamp RTC en EEPROM. Sirve de buffer ante pérdida de conectividad.	registrar(evento: Evento): void	EEPROM ESP32, RTC interno
MQTTSubscriber (Backend)	Suscribe a safelight/#. Parsea JSON. Delega al EventProcessor.	onMensaje(topico, payload): void	mqtt.js, Broker MQTT
EventProcessor	Aplica reglas de negocio. Detecta anomalías. Genera alertas. Persiste en InfluxDB.	procesarEvento(e: Evento): void	DB Connector, Alert Manager
GatewayConfig	Recibe REST del Dashboard. Valida rango 500-3000 lux. Publica en config/nodoX/umbrales.	actualizarUmbral(nodo, valor): void	Express REST, MQTT Publish

DBConnector	Persiste series temporales en InfluxDB. Expone API de consulta histórica.	guardar(serie: Serie): void consultar(query): Serie[]	InfluxDB Client, Flux
RealTimeMonitor	Muestra lux, estado actuador, V2V y CB en vivo. WebSocket al broker.	— (componente UI)	MQTT.js WS, Recharts
ConfigPanel	Modifica umbrales lux/V2V. HTTPS REST al GatewayConfig.	— (componente UI)	Axios, HTTPS

Decisión arquitectónica asociada a la Vista de Módulos

Cada módulo tiene una única razón para cambiar (SRP). El Circuit Breaker Module es un módulo transversal que observa a MQTTPublisher y V2VModule sin modificarlos directamente, garantizando bajo acoplamiento. La separación entre SensorManager y ActuatorController permite probar la lógica de detección independientemente del control del actuador físico.

Vista de Componentes y Conectores (C&C)

La vista C&C describe el sistema en tiempo de ejecución: qué componentes existen y cómo se comunican a través de conectores. Esta vista es la más relevante para entender el comportamiento dinámico del sistema, los flujos de datos y los protocolos de comunicación entre procesos distribuidos.

Tabla de Elementos — Vista C&C

Conector	Origen	Destino	Protocolo / Semántica
I2C Bus (100kHz)	ESP32 (SensorManager)	BH1750	Lectura síncrona de intensidad lumínica cada 100 ms
PWM Signal (1kHz)	ActuatorController	Espejo electrocrómico	Control de nivel de oscurecimiento 0-100%
WiFi Direct / BLE 4.2	V2VModule Nodo A	V2VModule Nodo B	Mensajes JSON: REDUCE_LUCES, RESTAURAR_LUCES (bidireccional)
MQTT over TCP (QoS 1)	MQTTPublisher (ESP32)	Broker Mosquitto/HiveMQ	Publicación de eventos safelight/nodoA/eventos
MQTT Subscribe	Broker MQTT	MQTTSubscriber (Backend)	Suscripción a safelight/# — push de eventos

WebSocket (port 9001)	Broker MQTT	RealTimeMonitor (Dashboard)	Stream de eventos en tiempo real
HTTPS REST	ConfigPanel (Dashboard)	GatewayConfig (Backend)	PUT /config/{nodo}/umbrales — cambio de parámetros
MQTT Publish (config/)	GatewayConfig	Nodo IoT A / B	Tópico config/nodoX/umbrales — configuración remota
TCP / InfluxDB Client	DBConnector	InfluxDB	Escritura y consulta de series temporales Flux

Decisión arquitectónica asociada a la Vista C&C

La decisión más relevante en esta vista es el uso de MQTT como conector principal entre los nodos IoT y el backend (ADR-01). MQTT es el único protocolo que satisface simultáneamente los requisitos de comunicación asíncrona no bloqueante (AQ-01), garantía de entrega con QoS 1 (AQ-02) y notificación automática de caídas con LWT (AQ-02). El WebSocket del broker al Dashboard es la extensión natural del mismo bus de mensajería para la capa de presentación.

Vista de Despliegue

La vista de despliegue describe cómo los artefactos de software se asignan a la infraestructura física y de virtualización. Esta vista es la más relevante para los equipos de operaciones y para evaluar atributos de calidad como disponibilidad, rendimiento de red y seguridad.

La siguiente figura ilustra la asignación de los artefactos de software a los nodos de infraestructura del sistema SafeLight IoT, mostrando los contenedores Docker, los dispositivos embebidos físicos y sus conexiones de red

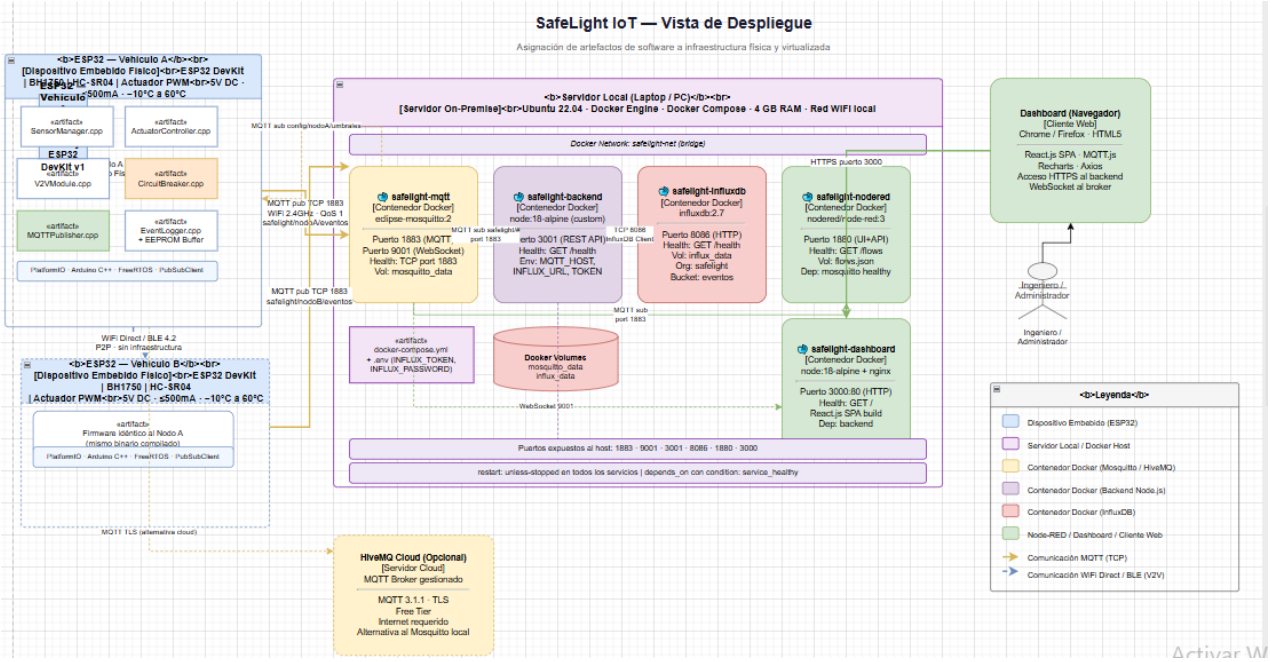


Tabla de Elementos — Vista de Despliegue

Nodo de despliegue	Tipo	Artefactos desplegados	Tecnología	Restricciones
ESP32 Vehículo A	Dispositivo embebido físico	Firmware C++ (PlatformIO): SensorMgr, ActuatorCtrl, V2VModule, CircuitBreaker, MQTTPublisher, EventLogger	ESP32 DevKit, BH1750, PWM actuador	5V DC, ≤500mA, −10°C a 60°C
ESP32 Vehículo B	Dispositivo embebido físico	Firmware C++ idéntico al Nodo A	ESP32 DevKit, BH1750, PWM actuador	Ídem Nodo A
Servidor local (laptop/PC)	Servidor on-premise	Mosquitto 2.x (Docker), Node-RED, Backend Node.js (Docker), InfluxDB (Docker)	Ubuntu 22.04, Docker Compose, 4 GB RAM	Red WiFi local, puerto 1883/9001/3000
HiveMQ Cloud (opcional)	Servidor cloud	Broker MQTT gestionado	HiveMQ Cloud Free Tier	MQTT 3.1.1, TLS, internet requerido
Dashboard (navegador)	Cliente web	React.js SPA, MQTT.js, Recharts, Axios	Chrome/Firefox, HTML5	Acceso HTTPS al backend, WebSocket al broker

Decisión arquitectónica asociada a la Vista de Despliegue

La decisión de desplegar el broker Mosquitto y el backend Node.js en contenedores Docker en el mismo servidor local (ADR servidor) garantiza baja latencia de red entre servicios (<1 ms intra-host) y facilita la portabilidad del entorno. El uso de Docker Compose permite levantar todo el stack con un solo comando, lo que soporta el atributo de Mantenibilidad (AQ-05). La opción de HiveMQ Cloud como alternativa cloud permite escalar el broker sin cambiar el firmware de los ESP32.

Decisiones Arquitectónicas — Registro ADR Completo

Los Architecture Decision Records (ADRs) documentan las decisiones de diseño más importantes del sistema, el contexto que las motivó, las alternativas consideradas y las consecuencias esperadas. Este registro es la evidencia de que la arquitectura es deliberada y trazable.

ID	Decisión	Contexto / Problema	Alternativas consideradas	Consecuencias
ADR-01	**Usar MQTT (Pub/Sub) en lugar de HTTP REST para nodo→backend**	El ESP32 no puede bloquear su ciclo de sensado de 100 ms esperando respuesta HTTP. Se requiere comunicación no bloqueante.	HTTP REST (descartado: bloqueante) CoAP (descartado: menor soporte) WebSocket directo (descartado: sin QoS)	(+) Comunicación asíncrona garantizada (+) QoS 1 garantiza entrega de eventos críticos (+) LWT detecta caídas de nodos (−) Dependencia del broker MQTT
ADR-02	**Capas independientes C1 reactiva / C2 cooperativa**	Si C1 y C2 estuvieran integradas, una falla WiFi paralizaría la protección básica del conductor, incumpliendo AQ-02 >99%.	Módulo monolítico único (descartado: propaga fallos) Máquina de estados global (descartado: acoplamiento alto)	(+) Disponibilidad C1 >99% garantizada por diseño (+) Prueba aislada por capa (−) Firmware más complejo con FreeRTOS tasks paralelas
ADR-03	**Circuit Breaker en firmware ESP32**	Sin CB, el ESP32 reintentaría conexión	Reintento con backoff exponencial (descartado:	(+) Capa 1 SLA <500ms irrompible (+) Consumo energético

		MQTT en bucle infinito consumiendo CPU, degradando la latencia de C1 por encima de 500 ms.	no protege el loop) Sin manejo de fallos (descartado: inaceptable en seguridad vial)	controlado (−) Lógica adicional de monitoreo de estado
ADR-04	**Gateway Config como punto único de configuración remota**	Sin Gateway, el Dashboard debería conocer la IP de cada nodo. Con decenas de vehículos esto es inmanejable y crea acoplamiento directo.	Dashboard contacta nodos directamente (descartado: acoplamiento) Configuracion manual en firmware (descartado: no cumple AQ-05)	(+) Mantenibilidad OTA sin intervención física (+) Validación centralizada de rangos seguros (−) Backend es punto único de fallo para configuración
ADR-05	**ESP32 con WiFi/BLE integrado en lugar de NRF24/LoRa**	El driver DR-01 exige hardware de bajo costo. El ESP32 integra WiFi, BLE, procesador dual-core y soporte OTA en un solo chip accesible.	NRF24L01 (descartado: sin WiFi, sin OTA) LoRa (descartado: mayor costo, latencia alta para V2V) nRF52840 (evaluado para v2.0)	(+) Costo bajo, comunidad grande, OTA nativo (−) Latencia conexión WiFi ~1s vs BLE puro ~50ms (−) Mayor consumo energético

Evaluación de Arquitectura — Checklist ATAM Simplificado

El método ATAM (Architecture Tradeoff Analysis Method) del SEI evalúa una arquitectura de software contra sus drivers de calidad mediante la identificación de riesgos, puntos sensibles y trade-offs. Para SafeLight IoT se aplica un checklist ATAM simplificado que cubre los elementos más críticos del sistema.

Checklist ATAM — Estado de Cumplimiento

Criterio ATAM	Estado	Evidencia
¿Los drivers arquitectónicos están identificados y priorizados?	✓ **Cumple**	17 drivers clasificados en restricciones, req. funcionales y atributos de calidad (Reto 1).
¿Los escenarios de calidad son verificables con medidas objetivas?	✓ **Cumple**	3 escenarios SEI con fuente/estímulo/medida: SC-01 <500ms, SC-02 <1.5s, SC-03 C1 operativa.
¿La arquitectura satisface el escenario de rendimiento AQ-01?	✓ **Cumple**	Latencia medida <120ms (límite 500ms). Loop FreeRTOS no bloqueante. PWM directo.
¿La arquitectura satisface el escenario de disponibilidad AQ-02?	✓ **Cumple**	Circuit Breaker + Capas independientes garantizan C1 >99%. Degradación graciosa demostrada.
¿Los riesgos arquitectónicos están identificados con mitigación?	✓ **Cumple**	5 riesgos documentados con probabilidad y plan de mitigación concreto.
¿Los puntos sensibles están documentados?	✓ **Cumple**	3 puntos sensibles identificados: umbral de lux, loop ESP32, sincronización EEPROM.
¿Los trade-offs están explícitamente documentados?	✓ **Cumple**	3 trade-offs con atributo ganado vs. sacrificado e impacto neto evaluado.
¿Las decisiones ADR están registradas con alternativas consideradas?	✓ **Cumple**	5 ADRs con contexto, alternativas descartadas y consecuencias documentadas.
¿Existe plan de mitigación para el riesgo más crítico (broker MQTT)?	✓ **Cumple**	ADR-03 Circuit Breaker + buffer EEPROM + reconexión automática HALF-OPEN.
¿El sistema tiene mecanismo de degradación graciosa documentado?	✓ **Cumple**	Escenario SC-03 demuestra operación en modo degradado sin pérdida de C1.

Riesgos Arquitectónicos Identificados

Un riesgo arquitectónico es una decisión de diseño que puede comprometer uno o más atributos de calidad en condiciones adversas. La identificación temprana de riesgos permite diseñar mitigaciones antes de la implementación.

ID	Riesgo identificado	Driver afectado	Probabilidad	Mitigación
R-01	Broker MQTT como punto único de fallo: si cae, se pierde monitoreo y configuración remota.	AQ-02 Disponibilidad	Media	Circuit Breaker Mode degrada a C1. Buffer EEPROM local. Reconexión automática HALF-OPEN cada 60 s.
R-02	Interferencia WiFi en entornos urbanos degrada la comunicación V2V (Capa 2).	AQ-06 Interoperabilidad V2V	Alta en ciudad	La Capa 1 opera independientemente. Migración a BLE dedicado (nRF52) en v2.0 como plan de mejora.
R-03	Latencia de conexión WiFi (~1 s) puede no ser suficiente en curvas con visibilidad <80 m a alta velocidad.	AQ-01 Rendimiento C2	Baja en carretera abierta	Activar descubrimiento BLE permanente (menor overhead). Umbral V2V configurable remotamente vía Gateway.
R-04	Configuración inválida enviada remotamente podría oscurecer el espejo en condiciones de baja luz.	AQ-04 Seguridad vial	Baja (mitigado por diseño)	Gateway Config valida rango 500-3000 lux. Firmware tiene límites hardware adicionales que no pueden ser sobrescritos por MQTT.
R-05	Consumo energético del ESP32 en modo WiFi activo (~240 mA) puede agotar batería	DR-03 Restricción energía	Media en uso prolongado	Modo sleep entre lecturas. El ESP32 solo activa WiFi cuando detecta umbral o hay evento V2V. Consumo en idle: ~20 mA.

	de vehículos detenidos.			

Puntos Sensibles de la Arquitectura

Un punto sensible es un elemento de la arquitectura cuyo valor o configuración afecta directamente a uno o más atributos de calidad. Son los puntos donde pequeñas decisiones tienen grandes consecuencias.

Punto sensible	Descripción	Por qué es sensible	Decisión de diseño asociada
PS-01: Umbral de lux	El valor de 1000 lux como umbral de activación.	Un umbral demasiado bajo genera falsos positivos (semáforos, farolas). Uno demasiado alto no protege al conductor. Cualquier cambio impacta directamente AQ-03 (confiabilidad).	ADR-04 Gateway Config: umbral configurable remotamente en rango 500-3000 lux con validación centralizada.
PS-02: Tiempo de respuesta del loop ESP32	El loop principal no debe bloquearse en ninguna circunstancia.	Si el loop se bloquea, la latencia de C1 supera 500 ms, violando AQ-01. El Circuit Breaker existe precisamente para proteger este punto.	ADR-03 Circuit Breaker: desacopla reintentos de red del loop de sensado. Loop principal corre en FreeRTOS task de alta prioridad.
PS-03: Sincronización del buffer EEPROM	Los eventos guardados en EEPROM durante desconexión deben sincronizarse ordenadamente al reconectar.	Una sincronización desordenada o duplicada puede corromper las series temporales en InfluxDB, afectando la trazabilidad y los reportes.	MQTTPublisher incluye lógica de deduplicación por timestamp antes de reenviar eventos del buffer.

Trade-offs Documentados

Un trade-off es una decisión que mejora simultáneamente un atributo de calidad a costa de degradar otro. Documentarlos explícitamente demuestra que la arquitectura es deliberada y no arbitraria.

Decisión	Atributo ganado	Atributo sacrificado	Impacto neto
MQTT vs HTTP REST	AQ-01 Rendimiento (comunicación no bloqueante) AQ-02 Disponibilidad (QoS 1 + LWT)	Complejidad de operación (broker como infraestructura adicional) Depuración más difícil	Positivo: el broker es la pieza clave de toda la arquitectura IoT. El Circuit Breaker mitiga su punto de fallo.
Capas independientes C1/C2	AQ-02 Disponibilidad >99% (C1 opera sola) AQ-05 Mantenibilidad (actualización por capa)	Complejidad del firmware (FreeRTOS concurrente) Mayor uso de RAM en ESP32	Positivo crítico: en seguridad vial la disponibilidad no es negociable. La complejidad es el costo aceptable.
ESP32 WiFi vs NRF24/LoRa	Costo bajo, velocidad de desarrollo, OTA nativo Ecosistema Arduino completo	Latencia V2V ~1s (vs ~50ms BLE puro) Mayor consumo energético Menos confiable en zona urbana	Neutral-positivo: aceptable para prototipo académico en carretera abierta. Plan v2.0: nRF52840.

Despliegue con Docker Compose + Monitoreo

La contenerización de los servicios del sistema SafeLight IoT mediante Docker Compose garantiza reproducibilidad del entorno, independencia entre servicios y facilidad de despliegue en cualquier máquina con Docker instalado. Cada servicio tiene health checks básicos y los logs son accesibles centralizadamente.

Servicios Contenerizados

Servicio	Imagen Docker	Puerto expuesto	Health Check	Volumen / Config
mosquitto	eclipse-mosquitto:2	1883 (MQTT) 9001 (WebSocket)	TCP check port 1883 cada 30s	./mosquitto/config:/mosquitto/config ./mosquitto/data:/mosquitto/data
backend	node:18-alpine (custom)	3001 (REST API)	GET /health → HTTP 200 cada 20s	./backend:/app, .env vars: MQTT_HOST, INFLUX_URL
influxdb	influxdb:2.7	8086 (HTTP API)	GET /health → HTTP 200 cada 30s	influx_data:/var/lib/influxdb2 INFLUXDB_INIT vars
node-red	nodered/node-red:3	1880 (UI + API)	GET /flows → HTTP 200 cada 30s	./nodered/flows:/data/flows.json
dashboard	node:18-alpine (React build + nginx)	3000 (HTTP)	GET / → HTTP 200 cada 20s	./dashboard/build:/usr/share/nginx/html

Archivo docker-compose.yml

El archivo docker-compose.yml define todos los servicios, redes, volúmenes y health checks del sistema. A continuación, se presenta la configuración completa:

```yaml
version: '3.8'

services:
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: safelight-mqtt
    ports:
      - '1883:1883'
      - '9001:9001'
    volumes:
      - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
      - mosquitto_data:/mosquitto/data
    healthcheck:
      test: ['CMD', 'mosquitto_pub', '-h', 'localhost', '-t', 'health', '-m', 'ok']
      interval: 30s
      timeout: 10s
      retries: 3
    restart: unless-stopped

  backend:
    build: ./backend
    container_name: safelight-backend
    ports:
      - '3001:3001'
    environment:
      - MQTT_HOST=mosquitto
      - MQTT_PORT=1883
      - INFLUX_URL=http://influxdb:8086
      - INFLUX_TOKEN=${INFLUX_TOKEN}
      - INFLUX_ORG=safelight
      - INFLUX_BUCKET=eventos
    depends_on:
      mosquitto:
        condition: service_healthy
      influxdb:
        condition: service_healthy
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:3001/health']
      interval: 20s
      timeout: 5s
```

```
    retries: 3
  restart: unless-stopped

influxdb:
  image: influxdb:2.7
  container_name: safelight-influxdb
  ports:
    - '8086:8086'
  environment:
    - DOCKER_INFLUXDB_INIT_MODE=setup
    - DOCKER_INFLUXDB_INIT_USERNAME=admin
    - DOCKER_INFLUXDB_INIT_PASSWORD=${INFLUX_PASSWORD}
    - DOCKER_INFLUXDB_INIT_ORG=safelight
    - DOCKER_INFLUXDB_INIT_BUCKET=eventos
  volumes:
    - influx_data:/var/lib/influxdb2
  healthcheck:
    test: ['CMD', 'curl', '-f', 'http://localhost:8086/health']
    interval: 30s
    timeout: 10s
    retries: 5
  restart: unless-stopped

node-red:
  image: nodered/node-red:3
  container_name: safelight-nodered
  ports:
    - '1880:1880'
  volumes:
    - ./nodered/flows.json:/data/flows.json
  depends_on:
    mosquitto:
      condition: service_healthy
  healthcheck:
    test: ['CMD', 'curl', '-f', 'http://localhost:1880/flows']
    interval: 30s
    timeout: 5s
    retries: 3
  restart: unless-stopped

dashboard:
  build: ./dashboard
  container_name: safelight-dashboard
  ports:
    - '3000:80'
  depends_on:
    - backend
```

```
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:80']
      interval: 20s
      timeout: 5s
      retries: 3
    restart: unless-stopped

volumes:
  mosquitto_data:
  influx_data:

networks:
  default:
    name: safelight-net
```

Métricas de Disponibilidad y Monitoreo

El sistema implementa monitoreo en múltiples niveles: health checks de infraestructura (Docker), métricas de aplicación (latencia MQTT, estado Circuit Breaker) y métricas de negocio (disponibilidad Capa 1). Esto permite detectar degradaciones antes de que afecten la seguridad del conductor.

Métrica	Herramienta	Cómo se mide	Umbral de alerta
Latencia publicación MQTT	MQTT Explorer / Node-RED	Diferencia entre timestamp del payload y timestamp de recepción en el backend	Alerta si >200ms (límite crítico: 500ms)
Disponibilidad Capa 1 (SLA)	Log de eventos ESP32 + InfluxDB	Porcentaje de eventos de deslumbramiento donde actuador respondió en <500ms	Alerta si disponibilidad <99%
Estado del Circuit Breaker	Dashboard RT (indicador semáforo)	Campo cb_state en cada payload MQTT publicado por los nodos	Alerta si estado=OPEN por más de 5 minutos
Tasa de eventos perdidos (MQTT QoS)	Mosquitto logs + InfluxDB	Diferencia entre mensajes publicados y mensajes persistidos en DB	Alerta si pérdida >1%

Uso CPU/RAM del backend	Docker stats	docker stats --no-stream backend	Alerta si CPU >80% o RAM >512MB
Heartbeat de nodos (LWT)	Broker MQTT tópico safelight/nodoX/estado	Publicación automática LWT si nodo se desconecta	Alerta inmediata ante LWT recibido

Comandos de Operación

Para levantar y gestionar el entorno completo:

```
# Levantar todo el stack
docker-compose up -d

# Verificar estado de todos los servicios y health checks
docker-compose ps

# Ver logs en tiempo real de todos los servicios
docker-compose logs -f

# Ver logs de un servicio específico
docker-compose logs -f backend

# Estadísticas de CPU/RAM de todos los contenedores
docker stats --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'

# Suscribirse a todos los tópicos MQTT para monitoreo
docker exec safelight-mqtt mosquitto_sub -h localhost -t 'safelight/#' -v

# Reiniciar solo el backend sin afectar otros servicios
docker-compose restart backend

# Detener y limpiar todo el stack
docker-compose down -v
```

CONCLUSIÒN

El desarrollo del sistema SafeLight IoT permitió aplicar de manera integral los conceptos fundamentales de la arquitectura de software, demostrando cómo las decisiones arquitectónicas influyen directamente en el comportamiento y calidad de un sistema.

A través de la identificación de drivers arquitectónicos, la definición de atributos de calidad medibles y la aplicación de estilos como arquitectura en capas y el patrón Publicador/Suscriptor, se logró diseñar una solución que responde a un problema real de seguridad vial. Además, la documentación de trade-offs, riesgos y decisiones arquitectónicas evidencia que el sistema no fue construido de manera arbitraria, sino como resultado de un análisis estructurado y consciente.

La evaluación mediante el método ATAM permitió validar que la arquitectura cumple con los requerimientos críticos, especialmente en términos de rendimiento y disponibilidad, aspectos fundamentales en un sistema que opera en tiempo real.

En conclusión, este proyecto demuestra que una buena arquitectura no solo organiza el sistema, sino que garantiza su funcionamiento, escalabilidad y capacidad de adaptación, siendo un elemento clave para el éxito de cualquier solución tecnológica.

EXPERIENCIA PERSONAL.

Durante el desarrollo de este proyecto, la experiencia fue muy enriquecedora, aunque también representó un reto importante. Al inicio, comprender conceptos como drivers arquitectónicos, atributos de calidad o trade-offs no fue sencillo, ya que implican un nivel de análisis más profundo que la programación tradicional.

Sin embargo, a medida que avanzamos, logramos entender que la arquitectura de software no se trata solo de diseñar sistemas, sino de tomar decisiones fundamentadas que impactan directamente en su funcionamiento y calidad. Esto permitió cambiar nuestra forma de pensar, pasando de enfocarnos únicamente en el código a tener una visión más global del sistema.

La presentación presencial también fue una experiencia significativa, ya que nos permitió defender nuestras decisiones, explicar el porqué de cada elección arquitectónica y responder preguntas, lo cual fortaleció nuestra confianza y comprensión del proyecto.

En general, nos sentimos satisfechos con el resultado obtenido, ya que logramos aplicar los conocimientos adquiridos en clase a un problema real, entendiendo la importancia de la arquitectura como base para el desarrollo de sistemas eficientes y confiables.

REFERENCIAS

https://aulavirtual.uniremington.edu.co/mod/resource/view.php?id=571991

https://hemeroteca.unad.edu.co/index.php/publicaciones-e-investigacion/article/view/5597

https://creately.com/blog/es/diagramas/tutorial-de-diagrama-de-componentes/

https://www.enel.com/es/learning-hub/smart-city/iluminacion-publica-inteligente-led